Modelo de Domínio

Do entendimento do negócio à representação conceitual em UML

Conceitos, exemplos, passo a passo e reflexão

Por que modelar um sistema?

Modelar é reduzir ambiguidade antes que ela vire código, banco de dados ou retrabalho.

Sem um modelo

Cada pessoa entende o negócio de um jeito.

Termos iguais podem ter significados diferentes.

Regras importantes ficam escondidas em textos ou conversas.

Com um modelo

Criamos uma linguagem comum.

Tornamos entidades e relações visíveis.

Validamos o entendimento com usuários e equipe.

Pergunta-chave

Antes de perguntar “como vamos programar?”, pergunte:

“Quais conceitos existem neste negócio e como eles se relacionam?”

Conexão com a aula anterior: antes perguntamos “o que o sistema faz?”. Agora mudamos o foco para “o que existe no domínio?”.

O que é domínio?

No material-base, domínio é definido como a área de negócio observada.

O domínio é o universo do problema que queremos compreender.

Exemplos: saúde, biblioteca, cinema, e-commerce, universidade, locação de veículos.

Cada domínio possui seu próprio vocabulário e suas próprias regras.

No domínio Saúde, por exemplo, aparecem conceitos como Paciente, Médico, Consulta, Receita e Exame.

O modelo de domínio descreve esses conceitos e as relações relevantes entre eles.

Reflexão

Em um sistema de universidade, “Turma” e “Disciplina” são a mesma coisa? Por quê?

Modelo de domínio ≠ código

É um modelo conceitual: representa o negócio, não uma decisão de implementação.

Modelo de domínio

Conceitos do negócio

Atributos relevantes

Relacionamentos

Multiplicidades

Sem operações/métodos

Diagrama de classes de software

Classes de implementação

Métodos/operações

Tipos e detalhes técnicos

Padrões de projeto

Decisões de arquitetura

Exemplo

Domínio: Consulta possui data e hora.

Implementação: agendar(), cancelar(), salvar(), enviarNotificacao().

As operações pertencem ao design do software, não ao modelo conceitual.

Classes conceituais: como encontrá-las?

Uma classe conceitual representa uma ideia, coisa, pessoa, evento ou registro relevante para o domínio.

Procure substantivos no texto de requisitos, mas não transforme todo substantivo em classe.

Pergunte: precisamos guardar informações sobre isso? Esse conceito tem identidade ou participa de relações importantes?

Categorias frequentes: pessoas/papéis, objetos, eventos/transações, lugares, documentos/registros e conceitos organizacionais.

Exemplo Saúde: Paciente, Médico e Consulta são bons candidatos; “telefone” normalmente é atributo, não classe.

Use nomes no singular: Paciente, Consulta, Livro, Empréstimo.

Classe ou atributo?

Uma das decisões mais comuns na modelagem é saber quando um conceito merece uma classe própria.

Paciente
nome
CPF
dataNascimento
e-mail
telefone

Consulta
data
hora
motivo
histórico

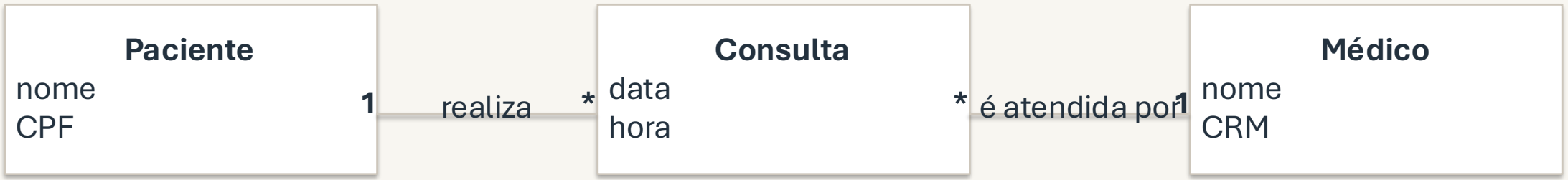
Médico
nome
CRM
telefone
especialidade

Regra prática: se o elemento apenas descreve outro conceito, tende a ser atributo. Se possui vida própria, regras ou relacionamentos, pode ser uma classe.

Exemplo: “endereço” pode ser simples atributo em um sistema pequeno, mas pode virar classe se houver múltiplos endereços, geolocalização, validação etc.

Relacionamentos: o que conecta os conceitos?

Uma associação indica que instâncias de duas classes podem estar conectadas no domínio.



Leia o diagrama como uma frase: “um Paciente realiza Consultas” e “uma Consulta é atendida por um Médico”.

O nome do relacionamento deve ajudar a explicar o significado da conexão.

Multiplicidade: quantos podem participar?

A multiplicidade informa o mínimo e o máximo de instâncias em cada extremidade da associação.

0..1

Zero ou um.

Relacionamento opcional.

Ex.: um funcionário pode ter 0 ou 1 vaga de estacionamento.

1

Exatamente um.

Relacionamento obrigatório.

Ex.: uma Consulta pertence a 1 Paciente.

0..*

Zero, um ou muitos.

Coleção opcional.

Ex.: um Cliente pode ter nenhum ou muitos Pedidos.

1..*

Um ou muitos.

Pelo menos um.

Ex.: um Pedido deve ter um ou mais Itens.

2..7

Faixa específica.

Entre 2 e 7.

Útil quando a regra do negócio impõe limites.

Como ler a multiplicidade sem se confundir

Sempre leia a multiplicidade que está do outro lado da associação.



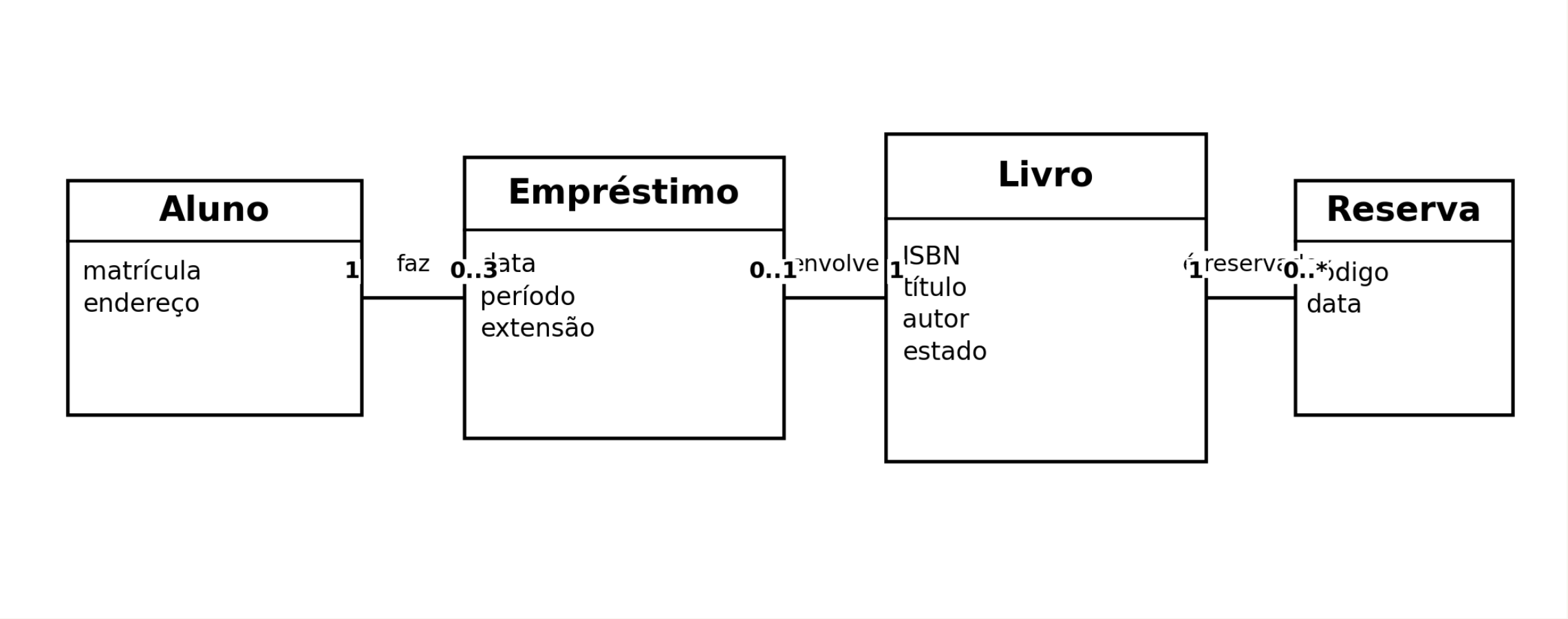
Pergunta 1: para UM Sócio, quantos Dependentes podem existir? → 0..*.

Pergunta 2: para UM Dependente, a quantos Sócios ele está associado? → 1.

Dica: formule as duas perguntas separadamente. Isso reduz muito os erros de cardinalidade.

Exemplo completo: biblioteca

O modelo original usa Aluno, Livro, Empréstimo e Reserva. Vamos interpretar o que cada conceito representa.



Processo para construir o modelo

O material propõe uma sequência simples; aqui ela está detalhada como roteiro de trabalho.

1. Leia o problema e delimite o domínio.
2. Liste classes candidatas a partir do conhecimento do negócio.
3. Releia requisitos/casos de uso e destaque substantivos relevantes.
4. Elimine falsos candidatos: atributos, ações, telas, tecnologias.
5. Nomeie classes no singular e crie a primeira versão.
6. Adicione relacionamentos e dê significado a eles.
7. Defina multiplicidades fazendo perguntas nos dois sentidos.
8. Acrescente apenas atributos relevantes para compreender o domínio.
9. Valide o modelo contra os requisitos e com alguém que conheça o negócio.

Do texto ao modelo: Clube Esportivo

Primeiro, extraímos conceitos e regras do enunciado antes de desenhar.

Conceitos candidatos

Sócio

Dependente

Categoria

Mensalidade

Atributos identificados

Sócio: cartão, nome, endereço, telefone, e-mail

Dependente: cartão, nome, parentesco, e-mail

Mensalidade: datas, valor, juros, quitada

Regras que viram relações

Sócio pode ter 0..* Dependentes.

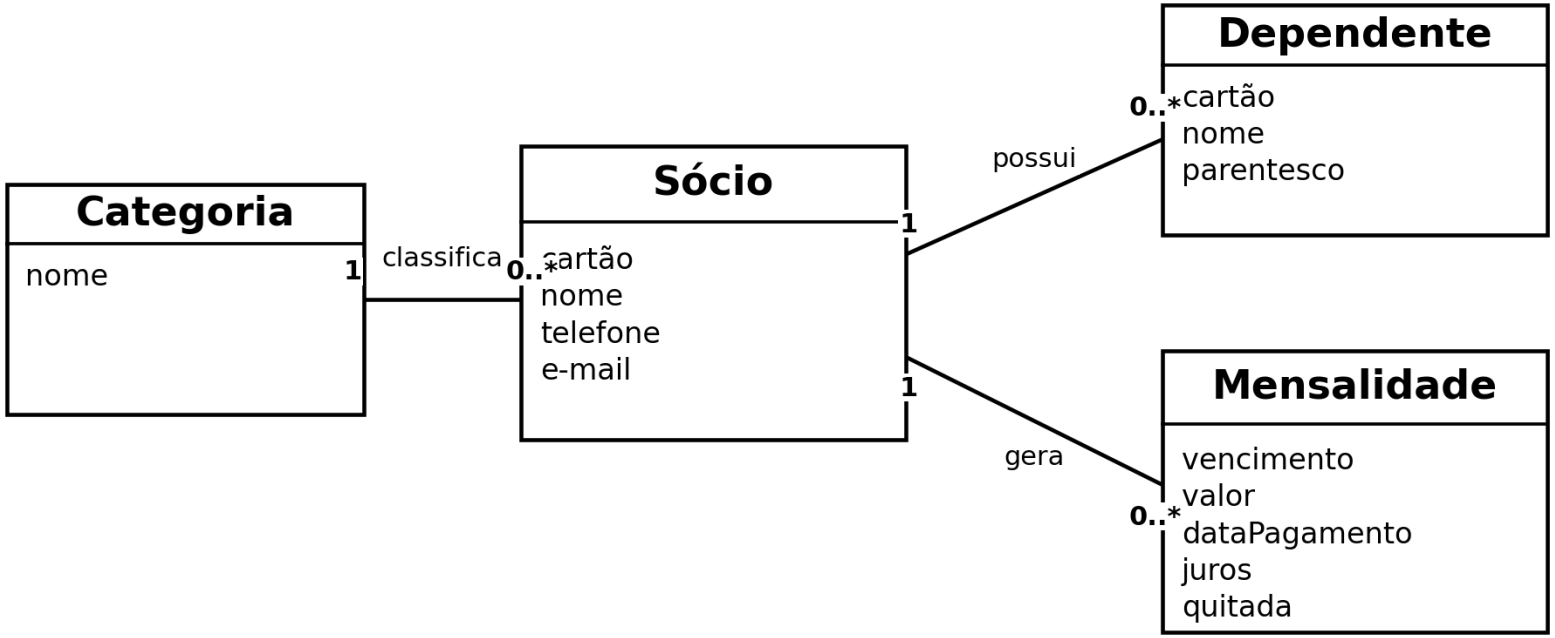
Dependente pertence a 1 Sócio.

Sócio pertence a 1 Categoria.

Sócio pode pagar muitas Mensalidades.

Clube Esportivo: modelo conceitual possível

O importante é conseguir justificar cada classe, atributo e multiplicidade a partir do enunciado.



Validação

Há alguma regra que não conseguimos representar?

Categoria precisa de outros atributos?

Mensalidade deve existir antes do pagamento?

“Pagar” é relação ou evento que merece outra classe?

Erros comuns em modelos de domínio

Um bom modelo não é o que tem mais caixas; é o que representa bem os conceitos relevantes.

Transformar verbos em classes sem necessidade: CadastrarCliente, EmitirRelatório, FazerLogin.

Colocar elementos de interface como classes: TelaLogin, BotãoSalvar, MenuPrincipal.

Adicionar tecnologia ao domínio: API, JSON, PostgreSQL, React — salvo se forem realmente conceitos do negócio.

Confundir atributo com classe ou duplicar a mesma informação em vários lugares.

Usar multiplicidade sem conseguir explicá-la em linguagem natural.

Modelar métodos/operações quando o objetivo ainda é compreender o domínio.

Copiar substantivos do requisito mecanicamente, sem avaliar sua relevância.

Exercício guiado: Sistema de Cinema

Use o enunciado do material-base e faça a modelagem em etapas.

1. Classes candidatas

Cinema

Sala

Filme

Sessão

Ingresso

Cliente

Funcionário

Compra?

2. Perguntas

Uma Sessão acontece em quantas Salas?

Uma Sala pode receber quantas Sessões?

Um Filme pode aparecer em quantas Sessões?

Um Ingresso pertence a uma Sessão?

Como representar a intermediação do Funcionário?

3. Regra importante

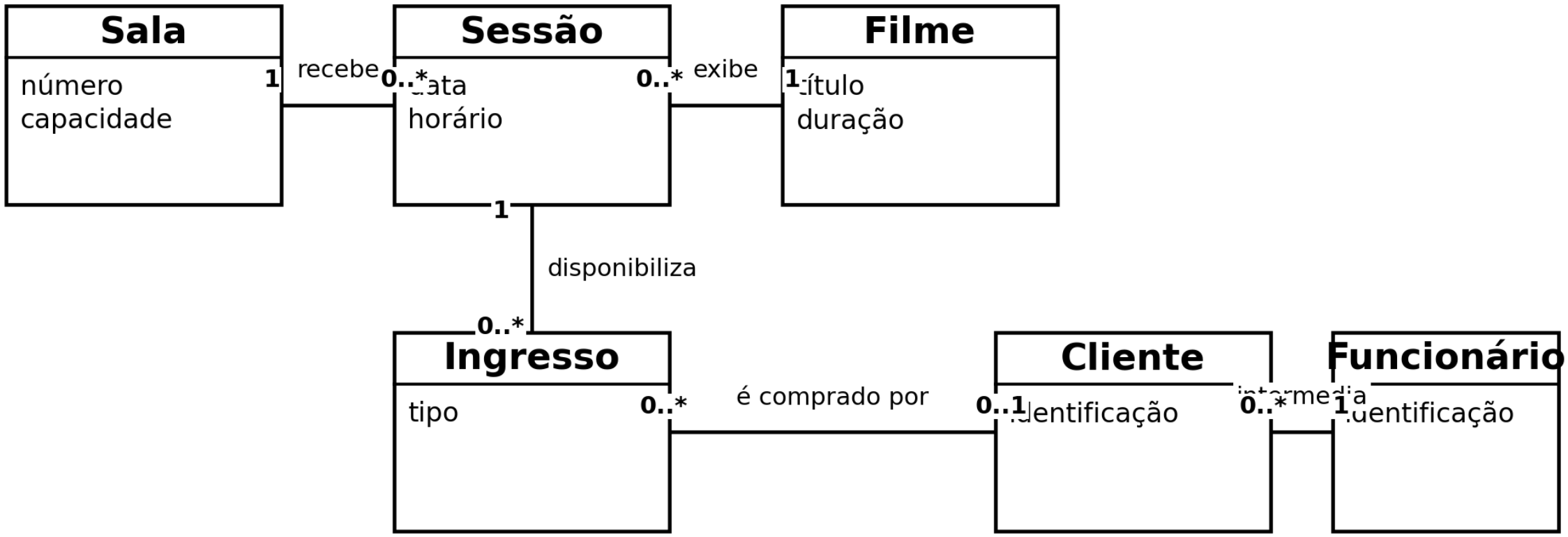
A quantidade máxima de ingressos de uma sessão é determinada pela capacidade da sala.

O cliente só pode comprar ingresso para sessão não encerrada.

Essas regras aparecem no modelo estrutural ou precisam ser registradas também como regras de negócio?

Uma solução possível para o Cinema

Não é a única solução. O objetivo é justificar as escolhas.



Perguntas de reflexão

Use estas questões para revisar conceitos ou abrir discussão em sala.

1. Todo substantivo encontrado em um requisito deve virar uma classe? Dê um contraexemplo.
2. Quando “Endereço” deveria ser atributo e quando deveria ser uma classe?
3. Qual a diferença prática entre 0..* e 1..*? Que regra de negócio muda?
4. Por que métodos como cadastrar(), excluir() ou pagar() normalmente não aparecem no modelo de domínio?
5. Em uma biblioteca, um “Empréstimo” é relacionamento ou classe? O que justifica sua resposta?
6. Se uma regra não aparece visualmente no diagrama, como garantir que ela não seja perdida?
7. Um modelo de domínio pode mudar mesmo sem mudar a tecnologia do sistema? Por quê?

Desafio rápido: encontre os problemas

Analise a proposta abaixo antes de revelar uma possível resposta.

Modelo proposto para e-commerce

Classes: Cliente, CPF, Pedido, TelaCheckout, Produto, BancoDeDados.

Cliente 1 — 1 Pedido.

Produto possui atributo comprar().

Pedido possui atributo cliente.

O que você questionaria?

CPF é conceito independente ou atributo?

TelaCheckout pertence ao domínio ou à interface?

BancoDeDados é negócio ou tecnologia?

Um Cliente realmente só pode ter 1 Pedido?

comprar() é atributo?

Se há associação Pedido–Cliente, precisamos duplicar cliente como atributo?

Síntese da aula

O modelo de domínio é uma ferramenta de entendimento e comunicação.

Domínio = área de negócio observada.

Modelo de domínio = classes conceituais + atributos relevantes + relações + multiplicidades.

A UML é usada como linguagem de representação, mas o foco é o significado do negócio.

A modelagem começa pelo entendimento dos requisitos, não pela tecnologia.

Multiplicidade traduz regras: quantos objetos podem ou devem estar associados.

O modelo deve ser validado continuamente e pode evoluir à medida que o entendimento do domínio melhora.

Reflexão

Se você tivesse que explicar seu modelo sem mostrar o diagrama, conseguiria descrevê-lo em frases claras?

Antes de começar: onde esta aula se encaixa?

A aula anterior e esta não repetem o mesmo modelo — elas observam o mesmo problema por perspectivas diferentes.

AULA 1 • MODELAGEM DE SISTEMAS

O que o sistema precisa fazer?

- Fronteira e contexto
- Atores
- Casos de Uso
- Especificação do comportamento

AULA 2 • MODELO DE DOMÍNIO

O que existe no negócio?

- Conceitos/classes
- Atributos
- Associações
- Multiplicidades

Não são duas modelagens concorrentes: são duas perspectivas complementares sobre o mesmo sistema.

Exemplo de amarração: Cinema

O mesmo cenário reaparece de propósito para mostrar a mudança de perspectiva.

MESMO PROBLEMA, DUAS PERGUNTAS

CASO DE USO

Cliente → Comprar ingresso

Foco: objetivo e interação com o sistema

→

MODELO DE DOMÍNIO

Cliente — Ingresso — Sessão — Filme — Sala

Foco: conceitos e relações do negócio

A pergunta muda: quais conceitos precisam existir para que a funcionalidade “Comprar ingresso” possa ser compreendida e realizada?